

1. Logika cyfrowa

Układ	Formuła / sygnatura	Opis
Sumatory		
Półsumator	$s = x \oplus y$ $c = x \& y$	Dodaje dwa bity. s to suma, c to bit przeniesienia (carry).
Pełny sumator (FA)	$s = x \oplus y \oplus ci$ $co = x \& y \mid ci \& (x \oplus y)$	Jak półsumator, ale przyjmuje też przeniesienie wejściowe ci .
Sumator n-bitowy	$n \times FA$	Kaskadowo wyjście C_i trafia na wejście kolejnego FA. Ostatnie co to Carry Out całości.

Układy kombinacyjne		
Dekoder	$n \rightarrow 2^n$	Wejście koduje liczbę k . Na wyjściu zapalony jest bit k .
Multiplexer	$2^n + n \rightarrow 1$	2^n bitów danych + n bitów sterujących S . Na wyjściu zapalony S -ty bit danych.
ALU	$A, B, f \rightarrow C$	Dekoder na bitach f wybiera operację np. $A + B$, multipleksowanie wyników bramkami AND/OR.

Układy sekwencyjne (pamięć)		
Przerzutnik S-R	S - set, R - reset	Przechowuje 1 bit (Q). S=1 ustawia $Q = 1$, R=1 zeruje, S=R=0 trzyma stan. Dwa sprzężone NOR -y.
Sterowany poziomem	aktywny gdy CLK = 1	Wejścia S / R zAND-owane z zegarem.
Sterowany zboczem	zbocze 1 $\rightarrow$ 0	Dwa przerzutniki poziomowe w kaskadzie. Stan przepisuje się w momencie zmiany zegara.

2. Typy danych

Typ	char	short	unsigned	int	long	float	double	void*
x86-64	1	2	4	4	8	4	8	8

3. Rozszerzanie, obcinanie i przesunięcia

$x \ll k$	$x \cdot 2^k$ (sgn./uns.)
$u \gg k$	$\lfloor u/2^k \rfloor$ - logiczne (zera)
$x \gg k$	arytmetyczne (kopia MSB), zaokr. do $-\infty$
$x/2^k$ do 0	$(x + (1 \ll (k-1))) \gg k$
Strength reduction	$x * 24 \rightarrow (x \ll 5) - (x \ll 3)$

4. Operacje i endianness

Negacja U2	<code>-x = ~x+1 ; -TMin=TMin , -0=0</code>	
Wykr. nadmiaru (<code>s=x+y</code>)	<code>((s^x)&(s^y))>>(w-1)</code>	
Maska / abs	<code>m=x>>31; abs=(x^m)-m</code>	
<code>c ? a : b</code>	<code>b ^ (~c & (a ^ b))</code>	
promocja w C: <code>unsigned</code> i <code>int</code> w jednym wyrażeniu/porównaniu < > ==) → <code>int</code> promowany do <code>unsigned</code> (bity bez zmian, -1 → Max).		
Schemat	Najniższy adres	<code>0x0123</code>
Big Endian	MSB	<code>[01][23]</code>
Little Endian	LSB	<code>[23][01]</code>

5. Zmiennoprzecinkowe (IEEE 754)

$V = (-1)^s \times M \times 2^E$ Bias = $2^{k-1} - 1$

Format	bity	s	exp (k)	frac (n)
float	32	1	8	23
double	64	1	11	52

Kategorie wartości		
Kategoria	exp	Własności
Znormalizowane	$!= 0 \dots 0$ $!= 1 \dots 1$	$E = \text{exp} - \text{Bias}$. $M = 1.\text{frac}$. Najgęściej ułożone wokół 0.
Zdenormaliz.	$== 0 \dots 0$	$E = 1 - \text{Bias}$. $M = 0.\text{frac}$. Reprezentują ± 0.0 (frac = 0) i liczby bardzo bliskie zera.
Specjalne	$== 1 \dots 1$	$\text{frac} == 0 \rightarrow \pm \infty$ (nadmiar, dzielenie przez 0). $\text{frac} != 0 \rightarrow \text{NaN}$ (np. $\sqrt{-1}$, $\infty - \infty$).

Zaokrąglenie GRS i Round-to-Even
Domyślny tryb to Round-to-Even (zapobiega błędowi statycznemu przy sumowaniu).

... x x G ! R S S S ... G - ostatni zachowany, R - pierwszy usunięty, S - 0R reszty		
Warunek	Ułamek	Akcja
R=0	< 0.5	W dół (odcięcie)
R=1, S=1	> 0.5	W górę (+1 do G)
R=1, S=0	$= 0.5$	Do parzystej: w górę gdy G=1 , w dół gdy G=0

Własności matematyczne i rzutowanie	
Brak łączności	$(a + b) + c \neq a + (b + c)$ - zaokrąglenia
Brak rozdzielności	$a(b + c) \neq ab + ac$
int $\rightarrow$ float	możliwa utrata precyzji (zaokrągła)
int $\rightarrow$ double	bezstratne ($52 > 32$ bity)
float/double $\rightarrow$ int	obcina do 0; poza zakresem lub NaN $\rightarrow$ TMin (0x80000000)

6. Rejestry x86_64

<code>%rax</code> <code>%eax</code> <code>%ax</code> <code>%ah</code> <code>%al</code>	<code>%rbx</code> <code>%ebx</code> <code>%bx</code> <code>%bh</code> <code>%bl</code>
<code>%rcx</code> <code>%ecx</code> <code>%cx</code> <code>%ch</code> <code>%cl</code>	<code>%rdx</code> <code>%edx</code> <code>%dx</code> <code>%dh</code> <code>%dl</code>
<code>%rsi</code> <code>%esi</code> <code>%si</code> <code>%sil</code>	<code>%rdi</code> <code>%edi</code> <code>%di</code> <code>%dil</code>
<code>%rbp</code> <code>%ebp</code> <code>%bp</code> <code>%bpl</code>	<code>%rsp</code> <code>%esp</code> <code>%sp</code> <code>%spl</code>
<code>%r8</code> <code>%r8d</code> <code>%r8w</code> <code>%r8b</code>	<code>%r9</code> <code>%r9d</code> <code>%r9w</code> <code>%r9b</code>

Uwaga: Rejestry od %r10 do %r15 używają schematu:
`%r10`, `%r10d`, `%r10w`, `%r10b`.

Podział ról rejestrów

<code>%rax</code>	Akumulator. Główny rejestr arytmetyczny. Przechowuje wartość zwracaną.
<code>%rdi</code> , <code>%rsi</code> , <code>%rdx</code> , <code>%rcx</code> , <code>%r8</code> , <code>%r9</code>	Kolejno: od 1. do 6. argumentu funkcji. Caller-saved.
<code>%rsp</code>	Wskaźnik stosu (SP). Wskazuje wierzchołek ramki.
<code>%rbp</code>	Wskaźnik bazy (BP). Początek ramki stosu. Callee-saved.
<code>%rbx</code> , <code>%r12</code> – <code>%r15</code>	Ogólnego przeznaczenia. Wszystkie callee-saved.
<code>%rip</code>	Wskaźnik instrukcji (Program Counter).

7. Tryby adresowania (operandy)

Tryb	Format	Obliczanie adresu
Natychmiastowy (Imm)	<code>\$Imm</code>	<code>Imm</code>
Rejestrowy (Reg)	<code>%Ra</code>	<code>%Ra</code>
Bezpośredni (Mem)	<code>Imm</code>	<code>MIImmI</code>
Pośredni (Mem)	<code>(%Rb)</code>	<code>MI%RbI</code>
Z przesunięciem	<code>D(%Rb)</code>	<code>MI%Rb + DI</code>
Skalowany (Ind/scaled)	<code>D(%Rb, %Ri, S)</code>	<code>MI%Rb+%Ri*S+DI</code>
Skalowany (baseless)	<code>(, %Ri, S)</code>	<code>MI%Ri * S</code>

Legenda: `Imm` / `D` to stała liczbowa, `%Ra` / `%Rb` to rejestr bazowy, `%Ri` to rejestr indeksowy, a `S` to skala (tylko: 1, 2, 4 lub 8).

8. Katalog instrukcji (AT&T)

Opcode	Efekt	Opis
<code>mov S, D</code>	$D \leftarrow S$	Kopiuje wartość z S do D.
<code>lea S, D</code>	$D \leftarrow \text{addr}(S)$	Oblicza adres S (bez czytania pamięci).
<code>add S, D</code>	$D \leftarrow D + S$	Dodawanie (ustawia flagi).
<code>sub S, D</code>	$D \leftarrow D - S$	Odejmowanie (ustawia flagi).
<code>imul S, D</code>	$D \leftarrow D * S$	Mnożenie liczb ze znakiem.
<code>sar / shl k, D</code>	$D \ll= k$	Przesunięcie w lewo (mnożenie przez 2^k).
<code>sar k, D</code>	$D \gg= k$	Arytmetyczne w prawo (dzielenie). Kopiuje znak.
<code>shr k, D</code>	$D \gg= k$	Logiczne w prawo. Dopełnia zerami.
<code>and / or S, D</code>	$D \leftarrow D \& / S$	Operacje bitowe (ustawiają flagi).
<code>xor S, D</code>	$D \leftarrow D \wedge S$	Często jako <code>xor %rax, %rax</code> do zerowania.

Uwaga o sufiksach: Instrukcje przyjmują sufiks określający rozmiar danych: `b` (8-bit), `w` (16-bit), `l` (32-bit), `q` (64-bit).
Np. `movl` kopiuje 32 bity (i automatycznie zeruje górną połowę 64-bitowego rejestru!).